React JS Notes

Introduction, Working, Flow, and Components

Level: Beginner
Topics: React Introduction, SPA, JSX, Virtual DOM, Components, Props, Rendering, and Project Structure

---

1. What Is React?

React is a **JavaScript library used to build user interfaces (UI)**, especially for web applications.

A user interface is the part of an application that users see and interact with.

Examples of UI Elements

- Buttons
- Forms
- Navigation bars
- Product cards
- Dashboards
- Login pages

React helps us build user interfaces using **reusable components**.

Example

A shopping website may contain the following components:

- `Navbar`
- `ProductCard`
- `Cart`
- `Footer`

Instead of writing the same code repeatedly, we create components and reuse them wherever required.

Important: React is a library, not a complete framework.

---

2. Why Do We Use React?

React provides several advantages.

1. Reusable Components

Write a component once and reuse it in multiple places.

2. Declarative UI

Describe what the UI should look like for the current data. React handles the necessary updates when the data changes.

3. Efficient UI Updates

React determines which parts of the DOM need to change and commits the necessary updates.

4. Data-Driven Interfaces

Display different content based on props, state, and other data.

5. Large Ecosystem

React can work with routing libraries, API tools, testing libraries, and many other packages.

6. Easier Maintenance

Breaking a large interface into smaller components makes the code easier to understand, test, and maintain.

---

3. What Is a Single-Page Application (SPA)?

A **Single-Page Application (SPA)** is a web application that can update its content without loading a completely new HTML document for every navigation.

Examples

- Gmail
- Interactive dashboards
- Many social media applications

How Does an SPA Work?

1. The browser loads the application's initial HTML document.
2. JavaScript loads and runs the application.
3. The application updates the displayed content as users interact with it.
4. A routing library can change the displayed page without requesting a new HTML document for every navigation.

Note: React is commonly used to build SPAs, but React does not automatically make every application an SPA. React can also be used in multi-page websites.

---

4. How Does a Traditional Website Work?

A simplified traditional website flow:

```
1. User opens a URL.
          |
          v
2. Browser sends a request to the server.
          |
          v
3. Server returns an HTML document.
          |
          v
4. Browser displays the webpage.
          |
          v
5. User navigates to another page.
          |
          v
6. Browser may request another HTML document.
```

This is a simplified explanation. Modern websites can use additional techniques such as caching, client-side JavaScript, and partial page updates.

---

5. How Does a React Application Work?

A simplified React application flow:

```
index.html
    |
    v
main.jsx
    |
    v
React Root
    |
    v
App Component
    |
    v
Child Components
    |
    v
UI Displayed in the Browser
```

Step-by-Step Explanation

1. The browser loads the application's HTML and JavaScript.
2. JavaScript loads and runs the React application.
3. `main.jsx` creates a React root.

4. React renders the root component, usually App.
5. Components return JSX describing the UI.
6. React calculates the required UI changes.
7. React commits necessary changes to the DOM.
8. When relevant props or state change, React can render the affected component tree again and update the DOM when needed.

Important: React does not normally replace the entire page for every update. It determines which DOM changes are necessary.

---

6. What Is JSX?

JSX stands for JavaScript XML.

It is a syntax extension that allows us to write HTML-like markup inside JavaScript.

Example

```
function App() {
    return <h1>Hello React!</h1>;
}

export default App;
```

The code looks similar to HTML, but it is JSX.

JSX is transformed by development build tools into JavaScript that React can use.

JSX Rules

Rule 1: Return one parent element

Incorrect:

```
function App() {
    return (
        <h1>Hello</h1>
        <p>Welcome to React</p>
    );
}
```

Correct:

```
function App() {
    return (
```

```
        <div>
            <h1>Hello</h1>
            <p>Welcome to React</p>
        </div>
    );
}
```

You can also use a Fragment (`<>...</>`) instead of adding an extra DOM element.

Rule 2: Use `className` instead of `class`

```
<h1 className="heading">Hello React</h1>
```

Rule 3: Close elements properly

```
<img src="photo.jpg" alt="Student" />
<input type="text" />
```

Rule 4: Use curly braces `{}` to insert JavaScript expressions

```
const name = "Amit";

function App() {
    return <h1>Hello, {name}!</h1>;
}
```

Rule 5: Component names should begin with a capital letter

```
function StudentCard() {
    return <h2>Student Details</h2>;
}
```

---

7. What Is the Virtual DOM?

The **DOM (Document Object Model)** represents a webpage as objects that JavaScript can access and manipulate.

React maintains an internal representation of the UI and uses it to determine the necessary updates.

This is commonly explained using the term **Virtual DOM**.

Example

Old UI:

```
<h1>Hello Amit</h1>
```

New UI:

```
<h1>Hello Riya</h1>
```

If the displayed name changes, React determines that the text needs to change and commits the necessary DOM update.

Simplified Process

1. React renders the component.
2. React calculates its new UI output.
3. React determines the necessary changes.
4. React commits the required DOM updates.

Important: The Virtual DOM is a useful learning concept, but React's rendering process involves more than simply comparing two trees. Rendering does not always result in a DOM change.

---

8. What Is a Component?

A **component is a reusable part of a user interface**.

Components help us divide a large application into smaller, manageable pieces.

Examples

- `Navbar`
- `Header`
- `Footer`
- `Button`
- `StudentCard`
- `ProductCard`
- `LoginForm`

Example

```
function Header() {
    return <h1>My Website</h1>;
}

function App() {
```

```
    return (
        <div>
            <Header />
            <p>Welcome to my website.</p>
        </div>
    );
}

export default App;
```

Explanation

- `Header` displays the heading.
- `App` combines the heading with the welcome message.
- `<Header />` tells React to render the `Header` component inside `App`.

---

9. Types of React Components

React supports function components and class components.

For beginners, **function components are the recommended starting point**.

A. Function Component

```
function Welcome() {
    return <h1>Welcome!</h1>;
}
```

B. Arrow Function Component

```
const Welcome = () => {
    return <h1>Welcome!</h1>;
};
```

Both examples define function components.

Difference Between the Two

Function Declaration	Arrow Function
Uses the `function` keyword	Uses `=>`
`function Welcome() {}`	`const Welcome = () => {}`

Function Declaration	Arrow Function
Can be called before its declaration because of function hoisting	A `const` variable cannot be accessed before initialization

Both forms can be used to create React function components.

---

10. Component Naming Rules

Follow these rules when creating components:

1. Component names must begin with a capital letter.
2. Use meaningful names.
3. Use PascalCase for component names.
4. Return valid React-renderable content.
5. Reuse components wherever appropriate.

Correct Names

```
function Navbar() {}
function StudentCard() {}
function UserProfile() {}
```

Incorrect as Custom Component Names

```
function navbar() {}
function studentcard() {}
```

Why?

In JSX, lowercase names are treated as built-in HTML elements, while capitalized names are treated as custom components.

For example:

```
<Navbar />
```

This renders the custom `Navbar` component.

```
<nav></nav>
```

This renders the built-in HTML `nav` element.

---

11. How to Use One Component Inside Another

A component can be used inside another component to build a larger interface.

Example

```
function Header() {
    return <h1>Student Portal</h1>;
}

function Footer() {
    return <p>Copyright 2026</p>;
}

function App() {
    return (
        <div>
            <Header />
            <p>Welcome, Student!</p>
            <Footer />
        </div>
    );
}

export default App;
```

Explanation

- `Header` displays the heading.
- `Footer` displays the footer text.
- `App` combines both components.
- `<Header />` and `<Footer />` are component usages.

This approach helps us organize a webpage into smaller sections.

---

12. What Is a Component Tree?

A **component tree** represents how components are nested inside other components.

Example

```
App
 |
```

```
|-- Navbar
|
|-- MainContent
|     |
|     |-- StudentCard
|     |
|     |-- StudentCard
|
|-- Footer
```

Explanation

- `App` is the parent component.
- `Navbar`, `MainContent`, and `Footer` are children of `App`.
- The two `StudentCard` instances are children of `MainContent`.

A component can contain other components, allowing us to organize complex interfaces into a clear hierarchy.

---

13. What Are Props?

Props are values passed from a parent component to a child component.

They allow the same component to display different data.

Example

```jsx
function StudentCard(props) {
    return <h2>Student: {props.name}</h2>;
}

function App() {
    return (
        <div>
            <StudentCard name="Amit" />
            <StudentCard name="Riya" />
        </div>
    );
}

export default App;
```

Output

```
Student: Amit
```

```
Student: Riya
```

The same `StudentCard` component displays different names because it receives different props.

Modern Syntax: Destructuring

Instead of writing `props.name`, we can extract `name` directly.

```
function StudentCard({ name }) {
    return <h2>Student: {name}</h2>;
}
```

Important Rules About Props

1. Props pass data from parent to child.
2. Props can contain strings, numbers, arrays, objects, functions, and other supported values.
3. Props are read-only.
4. A child component should not directly modify its props.
5. Props help make components reusable.

---

14. What Is Component Reusability?

Reusability means creating a component once and using it in multiple places.

Example 1: Reusing a Button

```
function Button() {
    return <button>Click Me</button>;
}

function App() {
    return (
        <div>
            <Button />
            <Button />
            <Button />
        </div>
    );
}
```

The same button component is rendered three times.

Example 2: Reusing a Button with Props

```jsx
function Button({ label }) {
    return <button>{label}</button>;
}

function App() {
    return (
        <div>
            <Button label="Save" />
            <Button label="Edit" />
            <Button label="Delete" />
        </div>
    );
}

export default App;
```

Output

Three buttons are displayed:

- Save
- Edit
- Delete

We do not need to create three separate components. We reuse one component and pass different labels through props.

---

15. What Happens When a Component Renders?

Rendering is the process through which React calculates the UI output of a component.

Simplified Rendering Process

1. React calls the component function.
2. The component returns JSX or other valid React content.
3. React calculates the resulting UI.
4. React commits the necessary changes to the DOM when required.

When Can a Component Render Again?

A component may render again when:

- Its state changes.
- Its parent renders and React renders the child as part of the update.
- Its context value changes.

A component does not necessarily cause DOM changes every time it renders.

Remember:

- **Render:** Calculate the UI output.
- **Commit:** Apply the necessary changes to the DOM.

---

16. Basic React Project Structure

A typical React project created with Vite may look like this:

```
my-react-app/
|
|-- node_modules/
|
|-- public/
|
|-- src/
|    |
|    |-- assets/
|    |
|    |-- App.jsx
|    |
|    |-- main.jsx
|    |
|    |-- style.css
|
|-- index.html
|
|-- package.json
|
|-- vite.config.js
```

Important Files and Folders

File or Folder	Purpose
`node_modules/`	Contains installed project dependencies
`public/`	Contains public assets served directly by Vite
`src/`	Contains the application's source code
`assets/`	Commonly stores images, fonts, and other assets
`App.jsx`	Commonly contains the main application component
`main.jsx`	Entry point that creates the React root
`style.css`	Contains CSS styles
`index.html`	HTML document containing the root element

File or Folder	Purpose
`package.json`	Contains project metadata, dependencies, and scripts
`vite.config.js`	Contains Vite configuration

Note: File names and folder structures can vary between projects.

---

17. Understanding `main.jsx`

The `main.jsx` file is commonly used as the entry point of a Vite React application.

Example

```
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
    <App />
);
```

Explanation

1. Import `createRoot`

```
import { createRoot } from "react-dom/client";
```

Imports the function used to create a React root.

2. Import the App component

```
import App from "./App.jsx";
```

Imports the main application component.

3. Find the root HTML element

```
document.getElementById("root")
```

Finds the HTML element whose ID is `root`.

4. Create a React root and render App

```
createRoot(document.getElementById("root")).render(
    <App />
);
```

Creates a React root and tells React to render the `App` component inside it.

Related `index.html` Code

```
<div id="root"></div>
```

React uses this element as the mounting point for the application.

---

18. Understanding `App.jsx`

The `App.jsx` file commonly contains the main application component and can combine smaller components.

Example

```
function Header() {
    return <h1>Welcome to React</h1>;
}

function App() {
    return (
        <div>
            <Header />
            <p>My first React application.</p>
        </div>
    );
}

export default App;
```

Explanation

- `Header` displays the heading.
- `App` combines the heading with a paragraph.
- `export default App` allows another file to import the component.
- `main.jsx` imports and renders `App`.

Application Flow

```
index.html
    |
    v
main.jsx
    |
    v
App.jsx
    |
    v
Header Component
    |
    v
UI Displayed in Browser
```

---

19. React Components vs Normal JavaScript Functions

React components are JavaScript functions, but they are used differently from ordinary utility functions.

Normal JavaScript Function

```
function add(a, b) {
    return a + b;
}

console.log(add(10, 20));
```

Output:

```
30
```

React Component

```
function Welcome() {
    return <h1>Welcome!</h1>;
}
```

The component returns JSX that React can render.

Key Differences

Normal Function	React Component
Commonly performs calculations or operations	Describes a part of the UI
Can return any JavaScript value	Must return valid React-renderable content when used as a component
Called like `add(10, 20)`	Rendered using `<Welcome />`
Does not automatically create UI	React uses its output to calculate UI

Important: During normal component composition, use `<Welcome />` instead of directly calling `Welcome()`. This allows React to manage component rendering and Hooks correctly.

---

20. Important React Terms

Term	Meaning
React	A JavaScript library for building user interfaces
Component	A reusable part of the UI
JSX	HTML-like syntax written inside JavaScript
Props	Data passed from a parent component to a child
State	Component-managed data that can change over time
Rendering	Calculating the UI output of a component
DOM	The browser's object representation of an HTML document
Virtual DOM	A common term for React's in-memory representation of UI
SPA	A web application that can update content without loading a new HTML document for every navigation
`main.jsx`	Common entry point that creates the React root
`App.jsx`	Common file for the main application component

---

21. Quick Revision

1. React is a JavaScript library for building user interfaces.
2. React applications are commonly built using components.
3. JSX allows HTML-like syntax inside JavaScript.
4. Component names begin with capital letters.
5. Components can be reused in multiple places.
6. Props pass data from parent components to child components.

7. Props are read-only.
8. State manages changing component data.
9. React renders components to calculate the UI.
10. React updates the DOM when necessary.
11. `main.jsx` commonly creates the React root and renders `App`.
12. `App.jsx` commonly contains the main application component.
13. A component tree shows the parent-child relationship between components.
14. A Fragment can group multiple JSX elements without adding an extra DOM element.
15. Rendering does not always mean the DOM changes.

---

22. Practice Tasks

Complete these tasks to strengthen your understanding of React components.

Level 1: Basic Components

Task 1: Create a `Welcome` component that displays:

```
Hello React!
```

Task 2: Create three components:

- `Header`
- `MainContent`
- `Footer`

Display all three inside `App`.

Task 3: Create a `Navbar` component that displays a website name and three navigation links.

Level 2: Props and Reusability

Task 4: Create a `StudentCard` component that accepts a student's name through props.

Task 5: Display three students using the same `StudentCard` component.

Example students:

- Amit
- Riya
- Rahul

Task 6: Create a reusable `Button` component that accepts a `label` prop.

Display three buttons:

- Save
- Edit

- Delete

Task 7: Create a `ProductCard` component that accepts the following props:

- Product name
- Price
- Category

Display at least three products.

Level 3: Component Composition

Task 8: Build a student dashboard using the following component structure:

```
App
 |
 |-- Navbar
 |
 |-- Sidebar
 |
 |-- Dashboard
 |     |
 |     |-- StudentCard
 |     |-- StudentCard
 |     |-- StudentCard
 |
 |-- Footer
```

Task 9: Create a `ProfileCard` component that accepts a name, course, and city through props.

Display at least three profile cards.

Task 10: Create a simple shopping webpage using these components:

- `Navbar`
- `ProductList`
- `ProductCard`
- `Footer`

Use props to display different product names and prices.

Final Summary

React helps developers build user interfaces using reusable components.

The basic concepts to understand first are:

1. React and its purpose
2. How a React application works

3. JSX syntax and rules
4. Components and component trees
5. Props and component reusability
6. Rendering and DOM updates
7. The roles of `main.jsx` and `App.jsx`

Once these concepts are clear, the next topics to learn are **Events, State (`useState`), Conditional Rendering, Lists, and the `useEffect` Hook**.